USB LED Control via D+ Pin Manipulation on Linux

A Study in USB 2.0 Test Modes, libusb, and PCIe Register Access

EE Department — IIT Hyderabad

Aarush

March 16, 2026

Abstract

This report documents the design and implementation of a system to control an LED using only a Linux laptop's USB data pin (D+), without any microcontroller or serial adapter. Two circuit variants are explored: one using a diode and one using only resistors. The software stack uses libusb to issue raw USB 2.0 Test Mode commands directly to the xHCI host controller, and a PCIe-level bus reset to exit the hardware lock imposed by test mode. The report explains the hardware design, the USB protocol internals, and the software implementation in full detail.

Contents

1 Project Overview **3**
1.1 Motivation and Goal . . . 3
1.2 Key Constraints . . . 3
1.3 Challenges . . . 3

2 USB Hardware Fundamentals **3**
2.1 USB Port Pin Layout . . . 3
2.2 How USB Data Signalling Works . . . 4

3 Circuit Design — Part 1: With Diode **4**
3.1 Circuit Schematic . . . 4
3.2 Component Roles . . . 4
3.3 Resistor Calculation . . . 4
3.4 How the Diode Circuit Works . . . 5

4 Software — Understanding libusb and USB Control Transfers **5**
4.1 What is libusb? . . . 5
4.2 What is a USB Control Transfer? . . . 5
4.3 Understanding bmRequestType . . . 6
4.4 SET_FEATURE and PORT_TEST . . . 6

4.5 Test Mode Selectors 7
4.6 wIndex Format for PORT_TEST 7

5 The Code — Part 1 (Diode Circuit) 7
5.1 led.c — Full Code with Explanation 7
5.2 gui.py — Full Code with Explanation 8
5.3 Why Standard Resets Fail 9

6 Circuit Design — Part 2: Without Diode 10
6.1 The Challenge 10
6.2 Why Direct D+ Connection Fails 10
6.3 The Working Circuit 10
6.4 How the Resistor-Only Circuit Works 11
6.5 Code Changes for Part 2 11

7 Build and Run Instructions 11

8 Results and Comparison 12

9 Conclusion 12

1 Project Overview

1.1 Motivation and Goal

The standard way to control an LED from a computer is through a microcontroller (such as an Arduino) which provides GPIO (General Purpose Input/Output) pins. These pins can be driven high or low freely by software.

A laptop's USB port does not expose GPIO pins. However, the USB data lines (D+ and D−) can be electrically manipulated under specific conditions defined in the USB 2.0 specification. This project exploits those conditions to achieve LED control directly from a Linux laptop with no additional hardware beyond a resistor, an LED, and optionally a diode.

1.2 Key Constraints

- No microcontroller or Arduino
- No USB-to-serial adapter
- Control must be fully software driven from Linux
- Minimum external components

1.3 Challenges

- USB 2.0 Test Modes are **one-way hardware latches** — the USB 2.0 specification (Section 7.1.20) explicitly states the only exit is a power-on reset of the device
- The keyboard and mouse share the same USB bus (Bus 001), so entering an illegal test state causes the xHCI controller to panic, freezing all input
- Standard software resets (libusb, sysfs driver rebind) cannot clear a silicon-level hardware lock

2 USB Hardware Fundamentals

2.1 USB Port Pin Layout

A standard USB Type-A connector has four pins:

Pin	Name	Voltage	Description
1	VBUS	+5V	Power supply. Always on when port is powered, regardless of device connection. Can supply up to 500mA (USB 2.0).
2	D−	0–3.3V	Differential data line (negative).
3	D+	0–3.3V	Differential data line (positive).
4	GND	0V	Ground reference.

2.2 How USB Data Signalling Works

USB 2.0 uses **differential signalling**. Data is not transmitted as a single voltage on one wire — instead, it is encoded as the *difference* between D+ and D−. This makes it resistant to noise.

The two meaningful electrical states are:

State	D+	D−	Meaning
J state	High (3.3V)	Low (0V)	Logic 1 / Idle
K state	Low (0V)	High (3.3V)	Logic 0 / Start of packet
SE0	Low (0V)	Low (0V)	Single-Ended Zero / Reset

> **Key insight:** VBUS is always powered. D+ can be forced to 0V via software using USB Test Mode. These two facts together make software-controlled LED switching possible.

3 Circuit Design — Part 1: With Diode

3.1 Circuit Schematic

```
      VBUS (5V) ----[R 300Ω]---- LED ----[Diode→]---- D+ (Pin 3)
GND (Pin 4) ------------------------------------------------------- GND
```

3.2 Component Roles

- **VBUS (5V):** Powers the LED. Has sufficient current capacity (up to 500mA) to drive the LED properly, unlike D+ which is a signal pin.
- **Resistor (300Ω):** Limits current through the LED to a safe value. Without it, the LED would draw too much current and burn out.
- **LED:** Red LED with forward voltage $V_f \approx 2\text{V}$.
- **Diode:** Acts as a current-steering element. Its orientation determines whether current flows toward D+ or is blocked.

3.3 Resistor Calculation

$$V_{available} = V_{VBUS} - V_{LED} = 5\text{V} - 2\text{V} = 3\text{V} \tag{1}$$

$$I_{target} = 10\text{mA} \quad \text{(standard LED operating current)} \tag{2}$$

$$R = \frac{V_{available}}{I_{target}} = \frac{3\text{V}}{0.01\text{A}} = 300\,\Omega \tag{3}$$

3.4 How the Diode Circuit Works

The diode's direction is: **anode toward LED, cathode (stripe) toward D+**.

LED ON — Normal State:

D+ is at its normal idle voltage (∼3.3V). The anode of the diode (connected to the LED side) is at approximately 2V (after the LED voltage drop). The cathode is at 3.3V. Since cathode > anode, the diode is **reverse biased** — it blocks current from flowing through it. Current therefore has no choice but to flow through the LED normally.

```
VBUS → R → LED → [Diode BLOCKED]   ← D+ (3.3V)
```

LED OFF — Test Mode Active:

D+ is forced to 0V by Test_SE0_NAK. Now the anode of the diode (at ∼2V) is **higher** than the cathode (at 0V). The diode becomes **forward biased**. Current takes the easier path through the diode toward D+ (which is at ground), completely bypassing the LED.

```
VBUS → R → [Diode CONDUCTS] → D+ (0V) → GND
```

D+ Voltage	Diode State	LED
∼3.3V (normal)	Reverse biased (blocked)	ON
0V (Test Mode)	Forward biased (conducting)	OFF

4 Software — Understanding libusb and USB Control Transfers

4.1 What is libusb?

libusb is an open-source C library that provides userspace applications direct access to USB devices, bypassing the normal operating system USB driver stack. Without libusb, communicating with USB hardware from a userspace program would require writing kernel-level driver code.

The abstraction stack when using libusb is:

Your Program (led.c)
libusb (userspace library)
Linux Kernel (usbcore)
xHCI Driver
xHCI Hardware Chip
Physical USB Port

4.2 What is a USB Control Transfer?

The USB 2.0 specification defines four types of data transfers. The most fundamental is the **Control Transfer**, used for sending commands and configuration requests to USB devices. It is the only transfer type guaranteed to be supported by every USB device.

According to **USB 2.0 Specification, Section 8.5.3** [1], a control transfer consists of three stages:

1. **Setup Stage:** An 8-byte SETUP packet is sent to the device containing the command details.
2. **Data Stage (optional):** Data is transferred if needed.
3. **Status Stage:** The device acknowledges completion.

The 8-byte SETUP packet has the following fields as defined in **USB 2.0 Specification, Table 9-2** [1]:

Field	Size	Offset	Description
bmRequestType	1 byte	0	Characteristics of the request (direction, type, recipient)
bRequest	1 byte	1	Specific request identifier
wValue	2 bytes	2	Request-specific parameter
wIndex	2 bytes	4	Request-specific parameter (often port/interface number)
wLength	2 bytes	6	Number of bytes in data stage

4.3 Understanding bmRequestType

The `bmRequestType` field is a bitmask. Its bits are defined in **USB 2.0 Specification, Table 9-2** [1] as follows:

Bits	Value Used	Meaning
Bit 7	0	Data direction: Host-to-Device (we are sending a command)
Bits 6:5	01	Type: Class request (hub-specific, not generic USB)
Bits 4:0	00011 (= 3)	Recipient: Other (specific port on a hub)

Combined: `0b00100011` = `0x23`

This value is specified in **USB 2.0 Specification, Section 11.24.2** [1] which defines hub class requests.

4.4 SET_FEATURE and PORT_TEST

The request `bRequest = 0x03` corresponds to **SET_FEATURE**, defined in **USB 2.0 Specification, Table 9-4** [1].

The value `wValue = 21` corresponds to **PORT_TEST**, defined in **USB 2.0 Specification, Table 11-17** [1] which lists hub class feature selectors:

Feature Name	Selector Value	Recipient
PORT_CONNECTION	0	Port
PORT_ENABLE	1	Port
PORT_RESET	4	Port
PORT_POWER	8	Port
PORT_TEST	**21**	**Port**

4.5 Test Mode Selectors

When SET_FEATURE is issued with PORT_TEST, the high byte of `wIndex` specifies *which* test mode to enter. These are defined in **USB 2.0 Specification, Table 7-22** [1]:

Selector	Name	D+	D−	Description
0	Reserved	—	—	Not used
1	Test_J	High	Low	Static J state
2	Test_K	Low	High	Static K state
3	**Test_SE0_NAK**	**0V**	**0V**	**Both lines to ground, NAK all IN tokens**
4	Test_Packet	—	—	Sends a specific test packet repeatedly
5	Test_Force_Enable	—	—	Forces port into enabled state

We use **Selector 3 (Test_SE0_NAK)** because it forces D+ to 0V, which is the electrical condition needed to turn the LED off.

4.6 wIndex Format for PORT_TEST

The format of `wIndex` when issuing PORT_TEST is defined in **USB 2.0 Specification, Section 11.24.2.13** [1]:

Bits	Content
15:8 (high byte)	Test Mode Selector
7:0 (low byte)	Port Number

In code: `wIndex = (3 << 8) | 9` means test mode selector 3 in the high byte, port 9 in the low byte. The `<<` is a left bit-shift operation:

$$3 = \texttt{0000 0000 0000 0011} \tag{4}$$

$$3 \ll 8 = \texttt{0000 0011 0000 0000} = 768_{10} \tag{5}$$

$$768 \mid 9 = \texttt{0000 0011 0000 1001} = 777_{10} \tag{6}$$

5 The Code — Part 1 (Diode Circuit)

5.1 led.c — Full Code with Explanation

Listing 1: led.c — Forces D+ to 0V via USB 2.0 Test Mode

```c
#include <stdio.h>
#include <stdlib.h>
#include <libusb-1.0/libusb.h>

int main(int argc, char *argv[]) {
    // Require exactly one argument (e.g. ./led 0)
    if (argc != 2) return 1;
```

```

    // Initialize libusb with default settings
    libusb_init(NULL);

    // Open the USB 2.0 root hub on Bus 001
    // VID 0x1d6b = Linux Foundation
    // PID 0x0002 = USB 2.0 Hub (the root hub)
    libusb_device_handle *dev_handle =
        libusb_open_device_with_vid_pid(NULL, 0x1d6b, 0x0002);
    if (!dev_handle) return 1;

    int port_number = 9; // The port the LED is connected to

    // Build wIndex: high byte = test selector (3), low byte =
       port number
    // As required by USB 2.0 Spec Section 11.24.2.13
    int wIndex_test = (3 << 8) | port_number;

    // Send SET_FEATURE(PORT_TEST) control transfer to the root
       hub
    // bmRequestType = 0x23: host->device, class request,
       recipient = port
    // bRequest      = 0x03: SET_FEATURE
    // wValue        = 21:   PORT_TEST feature selector (USB 2.0
       Spec Table 11-17)
    // wIndex        = wIndex_test: test mode 3 (SE0_NAK) on port
        9
    libusb_control_transfer(dev_handle, 0x23, 0x03, 21,
                            wIndex_test, NULL, 0, 1000);

    libusb_close(dev_handle);
    libusb_exit(NULL);
    return 0;
}
```

5.2 gui.py — Full Code with Explanation

Listing 2: gui.py — Mouse-driven LED control interface

```
import tkinter as tk
import os
import time
import subprocess

def led_off():
    # Calls the compiled C program which issues Test_SE0_NAK
    # This forces D+ to 0V, turning the LED OFF
    os.system("sudo ./led 0")

def led_on():
```

```
    # Standard USB resets fail because test mode is a silicon-level lock.
    # Solution: reset the xHCI controller at the PCIe bus level.

    # Step 1: Remove the xHCI controller from the PCIe bus entirely.
    # Writing "1" to the 'remove' sysfs file tells the kernel to
    # detach the device from the PCIe bus, resetting its silicon.
    subprocess.Popen(
        ["sudo", "tee", "/sys/bus/pci/devices/0000:00:14.0/remove"],
        stdin=subprocess.PIPE
    ).communicate(b"1")

    time.sleep(1)  # Wait for controller to fully disappear

    # Step 2: Rescan the PCIe bus.
    # The kernel rediscovers the xHCI controller, fresh with no test mode.
    # D+ returns to its normal idle state, LED turns back ON.
    subprocess.Popen(
        ["sudo", "tee", "/sys/bus/pci/rescan"],
        stdin=subprocess.PIPE
    ).communicate(b"1")

# Suppress kernel log spam during PCIe reset
os.system("sudo dmesg -n 1")

# Cache sudo password BEFORE keyboard freezes from the bus panic
os.system("sudo -v")

root = tk.Tk()
root.attributes('-topmost', True)  # Stay on top when keyboard is frozen
root.geometry("300x150")
root.title("LED Controller")

tk.Button(root, text="LED OFF", command=led_off,
          bg="red", width=20, height=3).pack(pady=5)
tk.Button(root, text="LED ON", command=led_on,
          bg="green", width=20, height=3).pack(pady=5)

root.mainloop()
```

5.3 Why Standard Resets Fail

When Test Mode is active, the xHCI host controller's PORTSC register (bits 30:28) is locked at the silicon level. The USB 2.0 specification, Section 7.1.20 [1], explicitly states:

> *"Test mode will be maintained until the device is power cycled (Vbus removed) or until the device is reset by a power-on reset."* — USB 2.0 Specification, §7.1.20

This means:

Reset Approach	Level	Works?
libusb_reset_device()	USB signal	No — silicon ignores it
sysfs unbind/rebind	Linux driver	No — driver can't fix locked silicon
authorized / deauthorize	Linux driver	No — same reason
PCIe remove + rescan	**PCIe bus**	**Yes — full chip reset**

The PCIe reset works because the xHCI controller is a PCIe device. Removing it from the PCIe bus and rescanning causes a full hardware reset of the chip, which is equivalent to a power cycle — exactly what the spec says is needed to exit test mode.

6 Circuit Design — Part 2: Without Diode

6.1 The Challenge

Removing the diode means we cannot use the current-steering technique from Part 1. We need a different approach to control the LED using only a resistor.

6.2 Why Direct D+ Connection Fails

The obvious alternative is to power the LED directly from D+:

```
D+ ----[R]---- LED ---- GND
```

This fails because D+ is a **signal line**, not a power rail. Although it nominally has 3.3V, the USB controller's output driver for D+ is designed to carry data signals at very low current. It cannot source enough current to drive an LED to visible brightness.

Testing with a 130 Ω resistor (theoretically correct for 10mA at 1.3V headroom) confirmed the LED did not glow.

6.3 The Working Circuit

The solution uses VBUS as the power source, with a single resistor feeding into D+, and the LED connected from D+ to GND:

```
VBUS (5V) ----[2.2kΩ]---- D+ ---- LED ---- GND
```

Component roles:

- **R (2.2kΩ):** Feeds current from VBUS onto the D+ node. The value must be high enough that the xHCI controller can overpower it and force D+ to 0V during Test Mode, but low enough to supply sufficient current through the LED to make it glow.
- **D+:** The switching node. Its voltage is controlled either by the pull-up resistor (LED ON) or by the xHCI controller in Test Mode (LED OFF).
- **LED:** Connected from D+ to GND. Glows when D+ is high, off when D+ is pulled to 0V.

6.4 How the Resistor-Only Circuit Works

LED ON — Normal State:

No test mode is active. Current flows from VBUS through the 2.2kΩ resistor, through D+, through the LED, and to GND.

$$V_{available} = V_{VBUS} - V_{LED} = 5\text{V} - 2\text{V} = 3\text{V} \tag{7}$$

$$I = \frac{3\text{V}}{2200\,\Omega} \approx 1.4\text{mA} \quad \text{(LED glows)} \tag{8}$$

LED OFF — Test Mode Active:

Test_SE0_NAK forces D+ to 0V. The xHCI controller's output driver actively pulls D+ to ground, overpowering the resistor pull-up. With D+ at 0V and the LED connected from D+ to GND, both ends of the LED are at 0V. There is no potential difference across the LED, so no current flows and the LED turns off.

> **Why R = 2.2kΩ?** The value was determined experimentally. Too low (e.g. 330Ω) and the xHCI driver cannot overpower the pull-up to hold D+ at 0V — the LED stays on. Too high (e.g. 1.8MΩ) and insufficient current flows through the LED for it to glow. At 2.2kΩ, the current is small enough for the xHCI driver to sink while keeping D+ firmly at 0V.

D+ State	Voltage at D+	Current through LED	LED
Normal (no test mode)	High (∼VBUS via R)	∼1.4mA	ON
Test_SE0_NAK active	0V	0mA	OFF

6.5 Code Changes for Part 2

The **led.c code is identical** — the same Test_SE0_NAK command is used to force D+ to 0V in both circuits. The hardware change is entirely in the circuit, not the software.

The **gui.py code is also identical** — the PCIe remove/rescan approach to exit test mode works the same way regardless of the LED circuit.

No code modifications are required when switching from the diode circuit to the resistor-only circuit.

7 Build and Run Instructions

Listing 3: Build and run

```
# Step 1: Compile the C program
gcc led.c -o led -lusb-1.0

# Step 2: Allow display access for sudo (required on Wayland/
    Hyprland)
xhost +local:

# Step 3: Run the GUI
```

```
sudo -v                    # cache sudo password first
sudo -E python3 gui.py     # -E preserves display environment
    variables
```

Or using the Makefile:

```
make
```

Makefile:

```
run:
    sudo -v
    sudo -E python3 gui.py
```

8 Results and Comparison

Aspect	Circuit 1 (Diode)	Circuit 2 (Resistor Only)
Components	VBUS, R (300Ω), LED, Diode, D+	VBUS, R (2.2kΩ), D+, LED, GND
OFF mechanism	Diode steers current to D+ (0V), bypassing LED	xHCI sinks D+ to 0V, eliminating voltage across LED
ON mechanism	Diode reverse-biased, current flows through LED	D+ pulled high by R1, current flows through LED
LED OFF state	Complete (zero current through LED)	Complete (zero voltage across LED)
LED ON state	Full brightness	Slightly dimmer (limited by R1+R2)
Code changes	—	None required
Robustness	High	Depends on xHCI driver strength vs R1

9 Conclusion

This project demonstrates that USB data pins can be repurposed as software-controlled electrical switches by exploiting USB 2.0 Test Modes. Two working circuits were implemented — one using a diode for current steering, and one using only resistors with a VBUS pull-up on D+. Both circuits use identical software: a libusb C program to issue the Test_SE0_NAK command (per USB 2.0 Specification Sections 7.1.20 and 11.24.2.13), and a PCIe-level bus reset to exit the hardware silicon lock imposed by test mode. The result is a fully functional LED controller driven entirely by Linux system calls, requiring no microcontroller or external interface hardware.

References

[1] USB Implementers Forum, *Universal Serial Bus Specification, Revision 2.0*, April 27, 2000. Available: https://www.usb.org/document-library/usb-20-specification

[2] libusb contributors, *libusb-1.0 API Reference*. Available: https://libusb.sourceforge.io/api-1.0/

[3] Intel Corporation, *eXtensible Host Controller Interface (xHCI) for Universal Serial Bus, Specification 1.2*, May 2019. Available: https://www.intel.com/content/www/us/en/products/docs/io/universal-serial-bus/extensible-host-controler-interface-usb-xhci.html